When the Map Hurts the Territory: Articulated Internal-Model Prompts Degrade Open-Weight LLM Sequence Prediction Across Scales

Mohamed A. Mabrok
m.a.mabrok@gmail.com

April 2026

Abstract

The Internal Model Principle (IMP) of Francis and Wonham [2] states that any controller that perfectly tracks a class of exogenous signals in steady state must contain, as a subsystem, a model of the signal generator. A natural translation of IMP to large language model (LLM) agents is the prompting hypothesis: *instructing an LLM to articulate its internal model of the environment before acting should improve closed-loop performance, especially as scale grows.* We test this hypothesis empirically on a 30-task sequence-prediction benchmark across three difficulty bands (easy, medium, hard) and four open-weight model scales (Qwen2.5-7B, Qwen2.5-14B, Qwen2.5-32B, Llama3.1-70B-q4) spanning roughly an order of magnitude in parameter count, under three matched conditions: (i) a reactive baseline that emits only the next integer, (ii) an IMP-style prompt that asks the model to state a one-line hypothesis about $f(k)$ before predicting, and (iii) a zero-shot chain-of-thought variant. Greedy decoding throughout. We find: (a) articulating an internal model *never* significantly helps, on any difficulty band, at any of the four scales; (b) on medium-difficulty tasks the hypothesis-prompt significantly hurts at every tested scale (paired-t on log-error, $p \in \{0.001, 0.010, 0.043, 0.025\}$; pooled Stouffer $p < 10^{-6}$); (c) the easy-band harm decays *monotonically* with scale (Spearman $\rho = -1.00$, Pearson $r = -0.998$, $p = 0.002$ over the four scales) but does not eliminate the medium-band harm, which persists at every scale; and (d) catastrophic single-cell failures (e.g. Fibonacci with reactive error 0.00 vs. hypothesis-prompt error 71.17 at 7B, geometric-2 with reactive 0.00 vs. hypothesis-prompt 159.50 at 32B) reveal a clear failure mode: the model emits a wrong stated rule and then deterministically computes forward from it. These results are inconsistent with the naive IMP-as-prompting reading and consistent with the meta-analytic finding of Sprague et al. [4] that chain-of-thought helps mainly on math/symbolic reasoning. We discuss implications for treating LLMs as controllers, list the experimental scope (one paradigm, greedy decoding, integer targets, single seed), and pre-register extensions.

Keywords: internal model principle, large language models, sequence prediction, prompt engineering, chain-of-thought, control theory, scaling.

1 Introduction

A theme of recent work in agentic AI is that LLMs increasingly play the role of *controllers* in closed loops: planners, tool-using agents, multi-agent committees, and online predictors. When a controller is asked to track an exogenous reference, control theory provides one of the few non-trivial sufficient/necessary conditions in the field — the Internal Model Principle, due to Francis and Wonham [2]. The IMP says, informally: *to achieve perfect asymptotic tracking of a class of disturbance or reference signals, the controller must replicate, as an internal subsystem, the dynamics that generate those signals.*

The IMP is a structural result about feedback architectures. It is silent about *how* the model is encoded. In a classical setting the model lives in a state-space realization of the controller. In a learned setting it could live in network weights and activations. In an LLM-agent setting it is tempting to read IMP as a prescription about prompts: *tell the LLM to write down its model of the generator before acting, and tracking will improve.* This reading is attractive because it is operational — one can implement "maintain an internal model" simply by editing the system prompt. It is also the reading that practitioners have implicitly been exploring for years under the banner of chain-of-thought [5, 3] and structured planning prompts.

This paper tests that reading directly. We construct a clean sequence-prediction game in which a hidden generator $f(k)$ produces an integer target at each step k, the LLM observes the past and emits a prediction $\hat{y}_k$, then the true y_k is appended to the history. We compare three system prompts: a *reactive* baseline that asks for one integer with no scaffolding, an *IM-Hypothesis* prompt that explicitly invokes the IMP and instructs the model to state a one-line hypothesis about $f(k)$ before predicting, and an *IM-CoT* zero-shot chain-of-thought prompt. We run the same 30-task benchmark, balanced across three difficulty bands, on three open-weight models spanning roughly an order of magnitude in scale.

The naive IMP-as-prompting prediction is straightforward: articulating an internal model should help — not necessarily uniformly, but at least in the regime where the underlying pattern is structured enough that having a model is informative. This prediction can be sharpened into three pre-registered hypotheses: (H1) IMP prompts hurt small models on tasks they can already solve reactively, but the harm decays with scale; (H2) IMP prompts help small models on hard tasks (replicating the chain-of-thought literature); (H3) the joint behaviour shows a significant *difficulty* $\times$ *condition* $\times$ *scale* interaction.

Our empirical findings refine H1, refute H2, and confirm H3 in a modified form:

1. Articulating an internal model *never* significantly helps, on any of the twelve *difficulty* $\times$ *scale* cells.
2. On *medium*-difficulty tasks the IM-Hypothesis prompt significantly hurts at all four scales (p values 0.001, 0.010, 0.043, 0.025; pooled Stouffer $Z = 5.06$, $p < 10^{-6}$). The harm survives an order of magnitude of scaling.
3. The easy-band harm decays *monotonically* with scale. The mean log-error gap (IM-Hypothesis − reactive) shrinks from +1.41 at 7B to +1.08 at 14B to +0.51 at 32B to 0.00 at 70B-q4. Spearman $\rho = -1.00$ ($p = 0$ at sample size 4), Pearson $r = -0.998$ ($p = 0.002$). This is a textbook clean scale-dependent decay — but it does not transfer to the medium band, where the harm persists at every scale.
4. The harm does not so much decay across the board as *migrate*: at 7B and 14B the IM-Hypothesis prompt damages *easy* tasks most (because reactive performance there is excellent and there is room to fall); at 32B and 70B reactive performance on easy approaches the floor, so the same prompt instead damages *medium* tasks where there is still slack.
5. Several individual cells exhibit a clean catastrophic failure: the reactive condition scores zero error while the IM-Hypothesis or IM-CoT condition produces large persistent error (e.g. Fibonacci on Qwen2.5-7B: 0.00 vs. 71.17; geometric-2 on Qwen2.5-7B and Qwen2.5-32B: 0.00 vs. ~ 160; geometric-2 on Qwen2.5-14B with IM-CoT: 0.00 vs. 384; squares-minus-k on Qwen2.5-32B with IM-CoT: 0.00 vs. 68.67). Inspection of the raw replies reveals a consistent mechanism — a wrong rule is committed in the hypothesis or reasoning line, and integer predictions are then deterministically computed forward from the wrong rule.

These observations are not a refutation of the IMP itself, which is a mathematical theorem about feedback systems with explicit state [2]. They are evidence that the *prompting reading* of IMP — "ask the LLM to externalize an internal model and use it" — is not a free lunch and can actively backfire. This is consistent with the broader picture emerging from the chain-of-thought literature: the recent meta-analysis of Sprague et al. [4] finds that CoT helps predominantly on

math/symbolic-execution tasks, with much smaller gains elsewhere. Our sequence-prediction setting is symbolic in nature, which is precisely the regime where one might expect articulation to help; that articulation *hurts* here is a non-trivial negative result.

The remainder of this paper is organized as follows. Section 2 restates the classical IMP, distinguishing mathematical content from prompting analogue. Section 3 positions the work within the chain-of-thought, multi-agent failure, and LLM-as-controller literatures. Section 4 gives the full benchmark description — exact prompts, exact tasks, exact metric, exact decoding — so the experiments are reproducible from the description alone. Section 5 reports the main table and the formal tests. Section 6 interprets the failure mechanism. Section 7 states the scope honestly: one task family, one decoding regime, integer targets, single seed, three open models. Section 8 lists pre-registered follow-up experiments that we believe are necessary before any general claim can be made.

2 The Internal Model Principle

We restate the IMP in its classical form to keep the prompting analogue under tight scrutiny. Consider an LTI plant P and an LTI controller K interconnected in a standard unity-feedback loop, with reference r, output y, and tracking error $e = r - y$. Suppose the reference belongs to a class of signals generated by an exosystem with state-space realization (A_e, B_e, C_e) — for example, A_e has poles at $\{0, \pm j\omega_1, \pm j\omega_2, \dots\}$ for steps and persistent sinusoids. Then [2, Theorem 1] establishes: *If the closed loop is internally stable and asymptotically rejects the disturbance class generated by (A_e, B_e, C_e), then the dynamic modes of the exosystem must be reproduced inside the loop* — that is, the controller (or some part of the loop) must contain a copy of the unstable modes of A_e. This is a *necessity* result: stability plus tracking implies the internal model.

Three observations bear on the prompting analogue.

The IMP is about modes, not language. The classical result fixes a representation (state-space transfer, Laplace domain) and the conclusion is about which poles appear in the loop. Translating "contains the model" to "writes down the model in words" is not implied by the theorem. The implicit model of an LLM, if any, lives in its weights and activations. Articulating that implicit model in tokens is a separate operation that need not be representation-preserving and may be lossy.

The IMP is about exact asymptotic tracking, not approximate prediction. The classical result is sharp under the condition of *zero* steady-state error. An LLM agent doing sequence prediction with finite horizon is not in the regime where this sharp condition is operative; the relevant criterion is average error, not exact tracking.

The IMP says "contains", not "commits to". An exogenous signal class admitting a high-dimensional internal model permits multiple realizations; the controller is free to maintain a distribution over candidates, condition on data, and refine. By contrast, an LLM that has just emitted "HYPOTHESIS: $f(k) = ak + b$ with $a = 5, b = 1$" has *committed* to one realization in the next-token sense: subsequent integer predictions will be conditioned on that text, and the autoregressive decoding will lock in the implied computation.

These three caveats already predict that the prompting analogue is not entitled to the strength of the original theorem. The empirical question is whether, despite these caveats, the analogue gives a useful heuristic. Our answer is: not in the setup tested.

3 Related Work

Chain-of-thought prompting. Wei et al. [5] demonstrate that prompting LLMs to produce intermediate reasoning steps before answering improves accuracy on arithmetic, commonsense, and symbolic-reasoning tasks — but chiefly on the largest models tested ($\geq$ 100B parameters in their original setup). Kojima et al. [3] simplify this to a zero-shot "Let's think step by step" prompt, the exact phrasing we use in our IM-CoT condition. The recent meta-analysis of Sprague et al. [4] surveyed over 100 CoT papers and ran controlled evaluations on 20 datasets across 14 models; their headline finding is that CoT gives strong gains primarily on math or logic, with much smaller gains on other tasks. Our work fits this picture: sequence prediction over deterministic integer generators is a math/logic-style task, exactly the regime where CoT should help; we observe instead that articulation hurts when implemented as our paper's IM prompts.

Multi-agent LLM failures. Beyond single-agent prompting, recent work investigates failure modes of multi-agent LLM systems. Cemri et al. [1] introduce MAST, a taxonomy of 14 failure modes across 7 frameworks built from rigorous human annotation of 1,600+ traces. Their Category I (system design issues) includes failures driven by prompt structure and role specification. Our finding contributes a single-agent specialization of that family: a system prompt that explicitly asks for an articulated internal model can drive accuracy down on a task family the same model would solve with no scaffolding.

Control-theoretic perspectives on LLMs. Several lines of recent work apply control-theoretic concepts to neural systems — stability, observability, feedback. Our paper is in this lineage but takes the opposite stance from much of it: rather than borrowing a control-theoretic guarantee and asserting that the LLM inherits it, we test, empirically and conservatively, whether one of the most foundational such guarantees translates *at the prompting level*. It does not. This contributes a data point for the broader question of which control-theoretic results have meaningful prompt-level analogues and which require working at the level of weights, activations, or training objectives.

4 Benchmark and Method

We describe the benchmark in full so the experiments are reproducible from this section alone. The implementation lives at `https://github.com/Minds-R-Lab/phase_margin_llm`, file `pipeline/experiments/run_imp_benchmark.py`.

4.1 Tasks

The benchmark contains 30 deterministic integer-valued generators $f\colon \mathbb{N} \to \mathbb{Z}$, balanced across three difficulty bands of 10 tasks each. Difficulty was hand-assigned according to the following criteria:

Easy: a no-reasoning baseline that simply echoes a recent observation tends to be approximately correct (constants, slow linear, low-period cycles).

Medium: identifying f requires explicit pattern recognition but, once identified, only one symbolic step is needed to compute the next value (quadratic, longer periods, piecewise, parity-modulated).

Hard: even with a correct hypothesis, the next-step computation is non-trivial (recurrences, polynomial in k, conditional rules, regime switches).

Table 1: The 30 benchmark tasks. $f(k)$ is the deterministic integer-valued generator; K is the trajectory length; the last ASYM steps are used for the asymptotic error. The description column is intentionally short. Tasks are addressed by their two-letter prefix in the rest of the paper.

ID	Difficulty	Generator $f(k)$	K	ASYM
e01_constant_seven	easy	7	14	6
e02_constant_zero	easy	0	14	6
e03_linear_2k+3	easy	$2k+3$	14	6
e04_linear_5k	easy	$5k$	14	6
e05_linear_desc_3	easy	$20-3k$	14	6
e06_period2_3_8	easy	cycle $[3,8]$	14	6
e07_period2_10_0	easy	cycle $[10,0]$	14	6
e08_period3_2_6_9	easy	cycle $[2,6,9]$	14	6
e09_period4_5_10_5_0	easy	cycle $[5,10,5,0]$	14	6
e10_mod5	easy	$k \bmod 5$	14	6
m01_quadratic_k2	medium	k^2	14	6
m02_quadratic_offset	medium	$(k-1)^2$	14	6
m03_period5_pi	medium	cycle $[3,1,4,1,5]$	14	6
m04_period6_e	medium	cycle $[2,7,1,8,2,8]$	14	6
m05_piecewise_v	medium	k if $k<5$ else $10-k$	14	6
m06_zigzag	medium	$k\cdot(-1)^k$	14	6
m07_step_floor3	medium	$\lfloor k/3 \rfloor$	14	6
m08_triangle_wave	medium	$\lvert(k \bmod 6)-3\rvert$	14	6
m09_linear_plus_period	medium	$k+3\cdot(k \bmod 2)$	14	6
m10_quadratic_bounded	medium	$-k^2+10k$	14	6
h01_fibonacci	hard	$f(0)=f(1)=1,\ f(k)=f(k-1)+f(k-2)$	14	6
h02_geometric_2	hard	2^k (capped to 4096)	12	4
h03_squares_minus_k	hard	$k(k-1)$	14	6
h04_triangle_numbers	hard	$k(k+1)/2$	14	6
h05_pi_digits	hard	first 16 digits of π, cycled	14	6
h06_e_digits	hard	first 16 digits of e, cycled	14	6
h07_cubic_mod100	hard	$k^3 \bmod 100$	14	6
h08_conditional_rule	hard	$2k$ if $k \bmod 3=0$ else $k+1$	14	6
h09_regime_switch	hard	cycle $[1,4]$ for $k<7$ then cycle $[5,2]$	14	6
h10_sum_two_periods	hard	$(k \bmod 3)+(k \bmod 5)$	14	6

The full task list is given in Table 1. All tasks use $K=14$ steps with the asymptotic error computed over the last 6, except `h02_geometric_2` which is capped at $K=12$ to avoid integer overflow in the prompt.

4.2 Conditions and Verbatim Prompts

Each task is run under three conditions, each defined by a single system prompt. We give the prompts verbatim because the central finding of this paper depends on subtle wording differences.

Reactive (baseline).

```
You are a sequence-prediction agent.  At each step you will be
shown all observations so far in the form
‘k=0: y=...; k=1: y=...; ...‘ and asked to predict the next value
y_k.  Reply with EXACTLY one integer on a line by itself.  No
commentary, no labels, no quotes.
```

IM-Hypothesis (the canonical IMP-as-prompt).

```
You are a sequence-prediction agent operating under the Internal
Model Principle.  At each step you will be shown all observations
so far and asked to predict the next value y_k.  You MUST first
state a one-line HYPOTHESIS about the function f(k) that generated
the observed values, then on a NEW LINE write your single integer
prediction for y_k.

Reply format (exactly two lines):
    HYPOTHESIS: <your one-line hypothesis>
    <integer prediction>
```

IM-CoT (zero-shot chain-of-thought variant).

```
You are a sequence-prediction agent.  At each step you will be
shown all observations so far and asked to predict the next value
y_k.  Let's think step by step: briefly reason about what rule
could be generating these values and why your candidate next
value is consistent with it.  After your reasoning, write your
single integer prediction for y_k as the LAST line of your reply,
by itself, with no labels or punctuation.  The integer on the
last line will be parsed automatically.
```

The user prompt is identical across all three conditions:

```
Observations so far: k=0: y=...; k=1: y=...; ...

Predict y_k.
```

The reactive condition uses `max_tokens = 96`, the IM-Hypothesis condition uses `max_tokens = 96`, the IM-CoT condition uses `max_tokens = 256` (because step-by-step replies are longer). Decoding is greedy (`temperature = 0`).

4.3 Metric

For each (task, condition, model) cell we run a single trajectory of K steps. The agent observes the history, emits a reply, and the last integer in the reply is parsed as the prediction $\hat{y}_k$. The trajectory is then advanced by appending the *true* y_k (not the prediction), so the agent sees the same history regardless of its own past errors — the loop is open in the prediction direction. The asymptotic error of the cell is the mean absolute error over the last ASYM steps:

$$E_{\text{asym}} = \frac{1}{\text{ASYM}} \sum_{k=K-\text{ASYM}}^{K-1} \left|\hat{y}_k - y_k\right|.$$

For statistical comparisons we work with $\log(E_{\text{asym}} + \varepsilon)$ with $\varepsilon = 0.5$ to keep the metric defined when one or both conditions score zero.

For each pair of conditions (a, b) and each difficulty band, we form the ten differences $\log(E_a + \varepsilon) - \log(E_b + \varepsilon)$ over the 10 tasks in that band and run a paired t-test. A negative t-statistic means condition b produces lower log-error on average; we report b as "hurts vs. a" when this is significant at $\alpha = 0.05$.

4.4 Models and Decoding

We test four open-weight model scales served via Ollama on a single H100 GPU:

Table 2: Mean asymptotic error per (model, difficulty, condition). Lower is better. Bold marks cells where a paired t-test on log-error against the same row's reactive baseline is significant at $p < 0.05$ in the *hurts* direction; † marks the same significance. No cell shows the help direction at $p < 0.05$.

Model	Difficulty	Reactive	IM-Hypothesis	IM-CoT
Qwen2.5-7B	easy	0.15	**6.08**†	0.57
	medium	1.58	**5.22**†	1.90
	hard	5.58	30.03	7.05
Qwen2.5-14B	easy	0.00	**1.72**†	**1.32**†
	medium	0.43	**9.22**†	**2.57**†
	hard	5.40	5.43	45.12
Qwen2.5-32B	easy	0.00	0.75	0.25
	medium	1.73	**2.48**†	2.30
	hard	5.57	20.85	**13.53**†
Llama3.1-70B-q4	easy	0.00	0.00 (tied)	0.02
	medium	0.37	**0.97**†	**1.83**†
	hard	6.77	8.03	11.25

- **Qwen2.5-7B-Instruct** (`qwen2.5:7b`) – 7.6B parameters, $\sim$ 190 s wall-time for the full benchmark.
- **Qwen2.5-14B-Instruct** (`qwen2.5:14b`) – 14.7B parameters, $\sim$ 580 s.
- **Qwen2.5-32B-Instruct** (`qwen2.5:32b`) – 32.5B parameters, $\sim$ 1140 s.
- **Llama3.1-70B-Instruct, q4_K_M** (`llama3.1:70b-instruct-q4_K_M`) – 70B parameters, 4-bit quantized, $\sim$ 9300 s.

The four scales span roughly an order of magnitude in parameter count. All four are open-weight; the 70B is the only quantized model in the set. Frontier-scale closed models (Claude, GPT, Gemini) are not tested in this paper; see Section 8.

Decoding is greedy in all cells (`temperature = 0`, single seed); this controls within-cell variance to zero, but at the cost of within-task variance estimation. Section 7 discusses this design choice and its consequences.

5 Results

We report (i) the main mean-error table by difficulty $\times$ condition $\times$ scale, (ii) paired t-tests at $\alpha = 0.05$, and (iii) the most-striking single-cell catastrophes that anchor the failure-mode interpretation.

5.1 Main Table

Table 2 reports the mean asymptotic error over the ten tasks in each difficulty band, broken out by model scale and prompting condition. Numbers are mean absolute error $|\hat{y} - y|$ on integer-valued targets, averaged over the last six steps of a 14-step trajectory and then over the ten tasks in the difficulty band.

The shape of the table is the central empirical content of this paper. Reading column-wise: (a) reactive is the strongest column on all three difficulty bands at 14B and 70B, and on easy and medium at 7B; (b) IM-Hypothesis is the worst column on 7B in all three bands and on 14B medium; (c) IM-CoT is the worst column on 14B hard and on 70B easy and medium. Reading row-wise: (d) both IM conditions inflate the mean across difficulty bands relative to reactive,

Table 3: Paired t-tests on log-error, reactive vs. each IM condition, by scale and difficulty. Each test has $n = 10$ tasks. The t-statistics shown are the negation of the log-difference test reported in the abstract: positive t here means the IM condition's log-error exceeds reactive's, i.e. IM hurts. At $\alpha = 0.05$, 11 of 23 testable cells are significant in the hurt direction; zero in the help direction; one cell (Llama-70B easy IM-Hyp) is not testable due to a zero-variance tie at the floor.

Model	Difficulty	vs. IM-Hyp		vs. IM-CoT	
		t	p	t	p
Qwen2.5-7B	easy	+3.25	**0.010**†	+1.38	0.20
	medium	+4.73	**0.001**†	+1.30	0.23
	hard	+2.20	0.055	+1.53	0.16
Qwen2.5-14B	easy	+3.38	**0.008**†	+2.44	**0.037**†
	medium	+3.28	**0.010**†	+4.58	**0.001**†
	hard	+1.37	0.20	+1.77	0.11
Qwen2.5-32B	easy	+1.85	0.097	+1.00	0.34
	medium	+2.35	**0.043**†	+1.13	0.29
	hard	+1.58	0.15	+2.52	**0.033**†
Llama3.1-70B-q4	easy	–	–(tied at floor)	+1.00	0.34
	medium	+2.68	**0.025**†	+2.77	**0.022**†
	hard	+0.60	0.56	+1.50	0.17

often by an order of magnitude or more. Reading by scale: (e) the easy column moves toward zero as scale grows for all three conditions, but reactive reaches the floor first.

5.2 Statistical Tests

Table 3 reports the paired-t statistics on $\log(E + 0.5)$ for each (scale, difficulty, IM condition) triple against the reactive baseline. $n = 10$ for every test.

Pooled significance on the medium band. The IM-Hypothesis condition hurts at every tested scale on the medium-difficulty band. Combining the four scale-specific two-sided p-values via Stouffer's method (with sign of t used to orient the z-transform) yields $Z = 5.06$, $p < 10^{-6}$. Stouffer requires independence across the combined tests; here the four runs are independent in their stochasticity (different models, separate cells), so the combined statement is sound.

Cross-scale trend on the easy band. The mean log-error gap of IM-Hypothesis vs. reactive on the easy band is $+1.41, +1.08, +0.51, 0.00$ across Qwen-7B, 14B, 32B and Llama-70B-q4 respectively. Treating these four points as a function of log(parameter count) gives Spearman $\rho = -1.00$ (perfect monotone decay; p formally zero at $n = 4$) and Pearson $r = -0.998$ ($p = 0.002$, two-sided). The same trend test for the medium band gives Spearman -0.80 ($p = 0.20$) and Pearson -0.92 ($p = 0.076$): a directionally consistent decay that is not yet significant at four points but falls outside the symmetry interval.

The IM-Hypothesis condition significantly hurts at all three scales on medium-difficulty tasks (p values 0.001, 0.010, 0.025). The IM-CoT condition does not hurt at 7B (the smallest model is too poorly-instructed to follow the CoT scaffolding tightly enough to lock in a specific wrong rule), but does hurt on easy and medium at 14B and on medium at 70B. The hard band shows no significant effect in either direction at any scale; reactive on hard is bad enough that ratio-based tests have low power and the t-statistics land in the $|t| < 2$ region.

The pattern that the harm *migrates* rather than *decays* is most clearly visible by reading the IM-Hyp p-value column down each scale block: 7B has its smallest p-values on easy/medium;

Table 4: Catastrophic single-cell errors with reactive at floor. The model demonstrably has the capability to predict the sequence (reactive error ≤ 0.5); inserting the IM scaffolding moves the error into the tens or hundreds. The phenomenon appears at every tested scale — including 32B — and on at least one catastrophic cell (`h02_geometric_2`) under IM-Hyp it reproduces almost numerically across the 7B and 32B scales.

Model	Task	Reactive	IM-Hyp	IM-CoT
Qwen2.5-7B	`e04_linear_5k`	0.00	41.67	0.00
Qwen2.5-7B	`h01_fibonacci`	0.00	71.17	0.00
Qwen2.5-7B	`h02_geometric_2`	0.00	160.00	0.00
Qwen2.5-7B	`m02_quadratic_offset`	0.00	11.83	3.83
Qwen2.5-14B	`m02_quadratic_offset`	0.00	66.17	7.83
Qwen2.5-14B	`h02_geometric_2`	0.00	0.00	384.00
Qwen2.5-14B	`h03_squares_minus_k`	0.00	5.33	21.00
Qwen2.5-32B	`m02_quadratic_offset`	0.00	8.83	16.50
Qwen2.5-32B	`m06_zigzag`	0.00	7.33	0.00
Qwen2.5-32B	`h02_geometric_2`	0.00	159.50	0.00
Qwen2.5-32B	`h03_squares_minus_k`	0.00	0.00	68.67
Qwen2.5-32B	`h04_triangle_numbers`	0.00	1.17	11.00

14B keeps the hurt on easy/medium; 70B loses the easy hurt only because its reactive is at the floor. In all three scales, the *medium* column is significantly hurt.

5.3 Mechanism: Single-Cell Catastrophes

The cleanest evidence about *why* the IM prompt hurts comes from individual cells in which the reactive condition achieves zero asymptotic error and the IM-Hypothesis condition does not. Such cells rule out the explanation "the model just cannot do the task." Table 4 lists the most-striking cases.

Inspection of the raw replies in these cells reveals a consistent mechanism: the model emits a wrong stated rule on the `HYPOTHESIS:` line — a slope-by-one error, an off-by-one quadratic, or a wrong recurrence — and then on the next line *deterministically computes forward from the wrong rule*, producing predictions that drift further from the true trajectory as k grows. The error is therefore not noise; it is the hypothesis-text *committing* the autoregressive decoder to a particular forward computation. When the same model is run under the reactive prompt on the same task, no such commitment is emitted and the implicit pattern-completion machinery (presumably weights tuned for next-token prediction on natural-text sequences that include integer patterns) suffices to predict correctly.

In control-theoretic terms: the IM-Hypothesis prompt does not expose the implicit internal model that the LLM already has; it forces the decoder to generate a *separate*, articulated internal model in tokens, which then becomes the operative model for downstream prediction. The articulated model is, in many cells, worse than the implicit one. This is the IM analogue of a classical control failure: the controller and the disturbance generator have the same modes, but the controller's *realization* of those modes is wrong, leading to negative interference rather than tracking.

6 Discussion

6.1 Why does the implicit model outperform the articulated one?

We offer one mechanism above; here we list three further candidate explanations and the evidence for and against each.

Hypothesis 1: tokenized commitment. The act of writing `HYPOTHESIS: <rule>` on line one creates a textual prefix on which subsequent integer generation is conditioned. If the rule is wrong, the integer is forced toward the rule. This matches the catastrophic cells in Table 4: the failures are *deterministic* given the wrong rule, not noisy.

Hypothesis 2: distributional shift in the prompt. The IM-Hypothesis prompt phrases the task as one in which the model must do hypothesis testing about $f(k)$. This may pull the distribution of model behaviour toward training data resembling mathematics homework or symbolic-regression problems — contexts where wrong intermediate symbolic claims are common. The reactive prompt, by contrast, puts the model in a vanilla-completion regime that is much closer to the next-integer-in-a-list pattern the model has seen in training data. We have not yet probed this directly; it is a concrete experiment for follow-up (Section 8).

Hypothesis 3: token budget pressure. IM-CoT gives more `max_tokens` than reactive, but its hurt is comparable to or weaker than IM-Hypothesis. This makes "the IM condition has *too few* tokens to express the right rule" an unlikely sole explanation. The IM-Hypothesis prompt also does not have a token-budget problem: it consumes only one extra line.

6.2 Comparison with the chain-of-thought literature

Wei et al. [5] demonstrate that CoT helps on complex multi-step reasoning, particularly for $\geq$ 100B parameter models. Our largest tested model is 70B-q4 (effectively smaller after quantization), so we are formally below the regime where their strongest gains were observed. Nevertheless, our paradigm (symbolic sequence prediction) is closer to the math/symbolic tasks where CoT does help in the original setup. Our finding — that CoT *does not* help and the explicit-hypothesis variant hurts — is therefore not contradicted by the original CoT paper but does narrow the regime of applicability further. In the language of Sprague et al. [4], our task family falls inside "math/symbolic" but the task structure (very short integer sequences, deterministic generators, fixed trajectory length) may not match the math problems on which CoT is empirically strong.

6.3 Implications for treating LLMs as controllers

A take-away for the LLM-as-controller programme: *the existence of an implicit internal model in an LLM does not license the prompt-level externalization of that model.* In our data, the implicit model embedded in network weights and activations consistently outperforms the model the same network articulates in tokens. This has design implications for agentic systems: prompting an LLM agent to "maintain a world model" or to "state its plan" before acting cannot be assumed to strengthen its control performance and may weaken it on deterministic-tracking tasks.

7 Limitations

We deliberately use this section to make the scope of our claim precise. All four limitations below are first-order and any one of them could reverse the qualitative finding under different choices.

One paradigm, integer sequences. The benchmark is single-agent sequence prediction with integer-valued, deterministic generators. Pattern types beyond what is in Table 1 are not tested. Continuous-valued, stochastic, or vector-valued tracking tasks may behave differently.

Greedy decoding, single seed. Setting `temperature = 0` eliminates within-cell variance but suppresses any benefit the IM-Hypothesis prompt might have under *self-consistency*-style decoding strategies, where multiple sampled hypotheses are aggregated. All single-seed claims should be treated as upper bounds on the help an IMP-style prompt could provide.

Four open-weight scales, one quantization. We have three Qwen2.5 sizes (7B, 14B, 32B) and one quantized Llama3.1 (70B, 4-bit). No frontier-scale proprietary model is included,

and the largest model is 4-bit quantized so its effective capacity is somewhere between 14B and 70B unquantized. The parameter-count span is roughly an order of magnitude, which is enough to detect a monotone scale trend in the easy band but not enough to extrapolate to two-orders-of-magnitude scaling.

Three prompt phrasings. The harm we report is specific to the exact prompts in Section 4.2. Other phrasings (e.g. "maintain an internal model" without the explicit `HYPOTHESIS:` line, or with explicit self-correction instructions) might give different effects.

We have not run robustness sweeps over any of these dimensions. For the level of generality the abstract claims, the data are not yet sufficient.

8 Extensions

We list, in priority order, the experiments needed to widen the claim from "hurts in this exact setup" to "hurts in general" or to characterise the regime in which IMP-style prompting could help.

E1. Add a non-quantized 70B and additional intermediate scales. Qwen2.5-32B has been added in this revision and confirms the migration prediction: easy-band harm decays monotonically (Spearman $\rho = -1.00$, Pearson $r = -0.998$, $p = 0.002$) while medium-band harm persists. The remaining gap is the unquantized 70B and ideally Qwen2.5-72B / Llama-3.1-405B for two-orders-of- magnitude span. Pre-registered prediction at larger scales: the medium-band harm continues to shrink approximately log-linearly, crossing into non-significance somewhere between 100B and 400B parameters.

E2. Self-consistency decoding. Re-run the IM-Hypothesis condition at `temperature = 0.7` with $N \in \{5, 10, 20\}$ sampled hypotheses per step, then either majority-vote the integer or pick the integer matching the most-frequent hypothesis. This tests whether *committing to one wrong rule* (our proposed mechanism) is the actual driver: if averaging across hypotheses recovers reactive performance, that is direct evidence for the commitment mechanism.

E3. Hypothesis quality probes. Programmatically classify the generated `HYPOTHESIS:` lines by correctness (does the stated rule match the true generator?) and condition the analysis on hypothesis correctness. Predicted: cells where the hypothesis is correct should match or beat reactive; cells where the hypothesis is wrong should account for the entire IM-Hyp−reactive gap.

E4. Counterfactual prompts. Test IM-Hypothesis variants with explicit invitation to revise the hypothesis when a prediction fails ("If your previous prediction was wrong, update the hypothesis"), and a self-consistency-aware variant that asks for the top-k hypotheses with weights. These probe whether the problem is the act of articulation or the absence of revision.

E5. Continuous-valued targets. Replace integer generators with continuous functions (sinusoids, slowly-varying nonlinear maps) and ask the agent to emit floating-point predictions. IMP in classical control is most natural over continuous signals; the prompting analogue may behave differently when the targets do not invite token-by-token integer commitment.

E6. Multi-agent setting. In MAS, the IMP analogue is that one agent plays the role of an explicit world-model maintainer and the other plays the controller. The MAST taxonomy of Cemri et al. [1] suggests this architecture frequently fails for reasons distinct from single-agent prompting. Running our benchmark with a two-agent (modeller, predictor) split, where the modeller's output is the predictor's prefix, would test whether the failure mode we observe is purely a same-context phenomenon or persists across agent boundaries.

E7. Architecture probes. The mechanism we propose ("tokenized commitment") is testable via training-time intervention: a model fine-tuned to emit hypotheses *conditional* on the integer prediction (i.e., generate the integer first, then articulate the rule) should not exhibit the forward-from-wrong-rule failure. This requires a small SFT run and is the most ambitious extension on this list.

E8. Frontier-model replication. Run the same benchmark on Claude Opus, GPT-4o, Gemini, and similar. If the harm disappears at very large scale, the headline becomes "Below frontier scale, IMP-as-prompt hurts." If it persists, the claim generalizes. This experiment is straightforward but incurs API costs and was out of scope for the open-weight infrastructure used here.

9 Why these results are useful

For practitioners building LLM agents. "Maintain an internal model" prompts are common in agentic-system templates. Our data suggest these prompts can degrade performance on deterministic tracking, sometimes catastrophically. The cheap pre-deployment check is to run the benchmark in this paper (or a domain-matched variant) under both reactive and articulated conditions and pick the better one. We expect this to flip designs in non-trivial cases.

For researchers borrowing from control theory. Our result is a calibration data point for the broader programme of mapping classical control concepts onto LLM agents. IMP is one of the strongest results in classical control; if its prompt-level prompts, observability scaffolds) deserve cautious empirical treatment before being treated as design principles.

For the prompt-engineering literature. We add a clean, reproducible cell to the negative-effects-of-CoT corpus [4] that targets a regime — short symbolic sequence prediction — where one would naively expect CoT to shine. The benchmark is small enough to run in ~ 10 minutes on a 7B model, which makes it suitable as an inclusion criterion for prompting papers that claim general benefits.

For multi-agent failure analysis. The harm we observe is the single-agent reduction of a class of failures the MAST taxonomy [1] catalogues at the system level. The benchmark gives a controlled probe for the single-agent component of those failures, which can then be composed back to study how they propagate when an articulated wrong model is consumed by a downstream agent.

10 Conclusion

We tested the prompting analogue of the Internal Model Principle on a 30-task sequence-prediction benchmark across four open-weight model scales spanning roughly an order of magnitude (7B, 14B, 32B, 70B-q4). On no (difficulty, scale) cell does articulating an internal model significantly help. On medium-difficulty tasks the explicit hypothesis prompt significantly hurts at every tested scale (pooled Stouffer $p < 10^{-6}$). The easy-band harm decays cleanly and monotonically with scale (Spearman $\rho = -1.00$, Pearson $r = -0.998$, $p = 0.002$), but it does not generalize: the medium-band harm persists at every scale, and catastrophic single-cell failures of the same kind appear at every scale, including 32B. Catastrophic single-cell failures expose the mechanism: the articulated rule *commits* the autoregressive decoder to a wrong forward computation that the same model would not have produced when prompted reactively. These results are inconsistent with the naive reading of IMP as a free-prompting heuristic for LLM agents and consistent with the broader picture that articulation does not uniformly improve LLM control behaviour. The benchmark, prompts, and analysis code are released so that each of the limitations and pre-registered extensions in Sections 7 and 8 can be addressed by independent replications.

Code and data. https://github.com/Minds-R-Lab/phase_margin_llm, file `pipeline/experiments/run_`

References

[1] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why Do Multi-Agent LLM Systems Fail? arXiv preprint arXiv:2503.13657, 2025.

[2] B. A. Francis and W. M. Wonham. The internal model principle of control theory. *Automatica*, 12(5):457–465, 1976. doi:10.1016/0005-1098(76)90006-6.

[3] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv preprint arXiv:2205.11916.

[4] Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. arXiv preprint arXiv:2409.12183, 2024. Published at ICLR 2025.

[5] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv preprint arXiv:2201.11903.